BEGINNER CLASSROOM GUIDE

Connecting a JavaScript Frontend to a FastAPI Backend

A step-by-step guide using the Deskflow ticketing project

HTML | CSS | JavaScript | FastAPI | JWT | SQLite

Frontend: http://127.0.0.1:5500
Backend: http://127.0.0.1:8000

What students will learn

By the end of this guide, students should understand how a browser sends HTTP requests to FastAPI, how JSON moves between the frontend and backend, and how a JWT token protects private endpoints.

Students will be able to:

- Explain the roles of HTML, CSS, JavaScript, and FastAPI.
- Start the frontend and backend as two separate programs.
- Read an API contract before writing frontend code.
- Send GET and POST requests with fetch().
- Convert JavaScript objects to JSON.
- Store a login token and send it in an Authorization header.
- Render API responses into an HTML page.
- Investigate failures with browser developer tools.

> **NOTE: The most important pattern is: browser event -> collect data -> call the API -> await the response -> update the page -> handle errors.**

1. The big picture

The frontend and backend are two separate programs. During development they run on the same computer, but they listen on different ports.

```
STUDENT
   |
   | clicks a button or submits a form
   v
HTML PAGE + JAVASCRIPT       http://127.0.0.1:5500
   |
   | HTTP request: URL, method, headers, JSON
   v
FASTAPI                     http://127.0.0.1:8000
   |
   | validation, authentication, permissions, database
   v
JSON RESPONSE
   |
   | JavaScript reads the response
   v
THE PAGE IS UPDATED
```

What each technology does

- HTML creates forms, buttons, tables, dialogs, headings, and page structure.
- CSS controls colours, spacing, responsive layout, and visual states.
- JavaScript listens for user actions and communicates with the API.
- FastAPI validates data, identifies users, enforces permissions, and accesses SQLite.
- JSON is the text format used to exchange structured data.
- HTTP status codes tell the frontend whether a request succeeded or failed.

> **IMPORTANT: Do not open index.html by double-clicking it. Serve the frontend with a local HTTP server. This gives it a normal origin and makes API behaviour easier to debug.**

2. Project structure

```
tickating_system/
|-- backend/
|   |-- app/
|   |   |-- main.py              # FastAPI app and CORS
|   |   |-- routes/
|   |   |   |-- auth.py          # register and login
|   |   |   |-- users.py         # current user
|   |   |   `-- tickets.py       # ticket endpoints
|   |   `-- ...
|   `-- service_desk.db
`-- frontend/
    |-- index.html                # page structure
    |-- styles.css                # visual design
    `-- app.js                    # API connection and behaviour
```

Most of the connection code is in frontend/app.js. The HTML gives JavaScript elements to work with through IDs such as login-form and ticket-table-body.

3. Start both programs

Step 1 - Start FastAPI

Open the first terminal and run:

```
cd /home/juliet/tickating_system/backend
source venv/bin/activate
alembic upgrade head
uvicorn app.main:app --reload
```

Test the backend by opening http://127.0.0.1:8000/docs. FastAPI's interactive Swagger page should appear.

Step 2 - Start the frontend

Open a second terminal and run:

```
cd /home/juliet/tickating_system/frontend
python3 -m http.server 5500
```

Open http://127.0.0.1:5500 in the browser.

> **NOTE: Two terminals are required because each server is a running program. Closing a terminal stops that server.**

4. Understand the API contract

An API contract tells the frontend which URL, HTTP method, request data, authentication, response data, and status codes to use.

The current Deskflow endpoints are:

```
POST  /auth/register          Create a normal user       Public
POST  /auth/login             Return a JWT token         Public
GET   /users/me               Return current user        JWT required
POST  /tickets                Create a ticket            JWT required
GET   /tickets                List/filter tickets        JWT required
GET   /tickets/{ticket_id}    Retrieve one ticket        JWT required
```

Registration request

```
POST /auth/register
Content-Type: application/json

{
  "full_name": "Ada Okafor",
  "email": "ada@example.com",
  "password": "SecurePass123!"
}
```

Login response

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "bearer"
}
```

The frontend must use the exact field names expected by FastAPI. For example, sending name instead of full_name produces a validation error.

5. Allow browser requests with CORS

CORS means Cross-Origin Resource Sharing. The browser treats ports 5500 and 8000 as different origins. Without permission from FastAPI, the browser blocks the frontend request.

The following code is in backend/app/main.py:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5500",
        "http://127.0.0.1:5500",
        "http://localhost:8080",
        "http://127.0.0.1:8080",
    ],
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

- allow_origins lists browser addresses permitted to call the API.
- allow_methods permits methods such as GET, POST, PATCH, DELETE, and OPTIONS.
- allow_headers permits Content-Type and Authorization headers.
- allow_credentials is false because this learning app sends a bearer header rather than a cookie.

IMPORTANT: CORS is enforced by browsers. A request may work in Postman or curl while the browser still reports a CORS error.

6. Connect HTML elements to JavaScript

The login form has an ID and its inputs have names:

```
<form id="login-form" novalidate>
  <input name="email" type="email" required />
  <input name="password" type="password" required />
  <button type="submit">Sign in</button>
</form>
```

JavaScript finds the form:

```
const elements = {
  loginForm: document.querySelector("#login-form"),
  registerForm: document.querySelector("#register-form"),
  ticketTableBody: document.querySelector("#ticket-table-body"),
};
```

Then JavaScript listens for submission:

```
elements.loginForm.addEventListener("submit", handleLogin);


HTML id="login-form"
        -> querySelector()
        -> submit event
        -> handleLogin()
```

7. Build one reusable API helper

Without a helper, every feature would repeat fetch(), headers, token handling, JSON parsing, and errors. The frontend puts that shared work in apiFetch().

7.1 Store the backend address

```
const API_BASE_URL = "http://127.0.0.1:8000";
```

Calling apiFetch("/auth/login") creates the complete URL http://127.0.0.1:8000/auth/login.

7.2 Build the headers

```
const headers = new Headers(options.headers || {});

if (options.body && !headers.has("Content-Type")) {
  headers.set("Content-Type", "application/json");
}

if (state.token) {
  headers.set("Authorization", `Bearer ${state.token}`);
}
```

- Content-Type tells FastAPI that the request body is JSON.
- Authorization identifies the signed-in user on protected endpoints.
- The helper adds the JWT automatically whenever one exists.

7.3 Send the request

```
response = await fetch(`${API_BASE_URL}${path}`, {
  ...options,
  headers,
});
```

await pauses this async function until the server answers. It does not freeze the whole browser.

7.4 Read JSON and detect errors

```
const payload = contentType.includes("application/json")
  ? await response.json()
  : null;

if (!response.ok) {
  throw new ApiError(
    getApiMessage(payload, response.status),
    response.status,
  );
}

return payload;
```

IMPORTANT: fetch() does not reject 401, 404, or 422 responses automatically. The code must check response.ok.

Complete request pattern

```
try {
  const data = await apiFetch("/some-endpoint", options);
  // use successful data
```

```
} catch (error) {
  showToast(error.message, "error");
}
```

8. Connect registration

Step 1 - Stop normal form submission

```
event.preventDefault();
```

Without this line the browser reloads the page, and JavaScript loses control of the request.

Step 2 - Check browser validation

```
if (!form.checkValidity()) {
  form.reportValidity();
  return;
}
```

Step 3 - Read the form values

```
const formData = new FormData(form);

const registration = {
  full_name: formData.get("full_name").trim(),
  email: formData.get("email").trim(),
  password: formData.get("password"),
};
```

The names must match the HTML input names and the Pydantic schema fields.

Step 4 - Send POST /auth/register

```
await apiFetch("/auth/register", {
  method: "POST",
  body: JSON.stringify(registration),
});
```

JSON.stringify() converts a JavaScript object into JSON text for the request body.

Step 5 - Update the interface

```
form.reset();
elements.loginEmail.value = registration.email;
switchAuthTab("login");
showToast("Account created. You can sign in now.");
```

Registration does not log the user in because this endpoint returns a user profile rather than a JWT.

9. Connect login and JWT authentication

Step 1 - Send credentials

```
const response = await apiFetch("/auth/login", {
  method: "POST",
  body: JSON.stringify({
    email: formData.get("email").trim(),
    password: formData.get("password"),
  }),
});
```

Step 2 - Save the token

```
state.token = response.access_token;
localStorage.setItem("deskflow_access_token", state.token);
```

- state.token makes the token available to the current page.
- localStorage keeps it after a refresh or browser restart.

> **IMPORTANT: localStorage is simple for this learning project. Production applications must carefully prevent cross-site scripting; secure cookie-based designs are also common.**

Step 3 - Send a protected request

```
state.user = await apiFetch("/users/me");
```

Because a token exists, apiFetch adds this header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

FastAPI decodes the token, reads the user ID, loads that user from the database, and returns the safe profile.

Step 4 - Display the user

```
elements.userName.textContent = state.user.full_name;
elements.userRole.textContent = state.user.role;
```

Step 5 - Logout

```
state.token = null;
state.user = null;
localStorage.removeItem("deskflow_access_token");
```

Logout removes the local token. Future protected requests no longer contain an Authorization header.

10. List, filter, and paginate tickets

Step 1 - Build query parameters

```
const query = new URLSearchParams({
  page: String(state.page),
  page_size: String(state.pageSize),
});
```



```
if (state.status) query.set("status", state.status);
if (state.priority) query.set("priority", state.priority);
```

Selecting open and high priority creates:

```
GET /tickets?page=1&page_size=8&status=open&priority=high
```

Step 2 - Request one page

```
const ticketPage = await apiFetch(`/tickets?${query}`);
```

The response contains items, page, page_size, and total.

```
{
  "items": [
    {
      "id": 4,
      "title": "Cannot access student portal",
      "priority": "high",
      "status": "open",
      "owner_id": 1
    }
  ],
  "page": 1,
  "page_size": 8,
  "total": 1
}
```

Step 3 - Render the ticket rows

```
elements.ticketTableBody.innerHTML = items
  .map(ticketRowTemplate)
  .join("");
```

map() turns every ticket object into an HTML row. join("") combines all the rows into one string.

Step 4 - Calculate the number of pages

```
const totalPages = Math.max(1, Math.ceil(total / pageSize));
```

If 21 tickets are displayed 8 at a time, Math.ceil(21 / 8) returns 3 pages. Previous and Next change state.page and load the dashboard again.

11. Create a ticket

The create form collects only title, description, and priority.

```
const ticket = await apiFetch("/tickets", {
  method: "POST",
  body: JSON.stringify({
    title: formData.get("title").trim(),
    description: formData.get("description").trim(),
    priority: formData.get("priority"),
  }),
});
```

> **NOTE: The frontend does not send owner_id or status. FastAPI obtains the owner from the JWT and starts every new ticket as open. This prevents impersonation and fake status values.**

After a successful request, the frontend:

- Clears the form.
- Closes the dialog.
- Returns pagination to page 1.
- Reloads the dashboard.
- Displays the new ticket number.

12. Retrieve one ticket

Each table button contains a ticket ID:

```
<button data-ticket-id="4">Open</button>
```

JavaScript reads the value and requests:

```
const ticket = await apiFetch(`/tickets/${ticketId}`);
```

It places the response into the details dialog:

```
elements.detailTitle.textContent = ticket.title;
elements.detailDescription.textContent = ticket.description;
elements.detailOwner.textContent = `#${ticket.owner_id}`;
```

> **NOTE: textContent displays text without treating it as HTML. This helps prevent script injection. When innerHTML is necessary, escape user-controlled values first.**

13. Handle errors clearly

Common status codes in this application are:

```
201  Created                  Registration or ticket succeeded
200  OK                       Login or retrieval succeeded
401  Unauthorized             Token missing, expired, or invalid
404  Not Found                Ticket missing or hidden
409  Conflict                 Email is already registered
422  Unprocessable Entity     Request failed validation
```

FastAPI validation errors are arrays. The helper converts them to readable messages:

```
if (Array.isArray(payload?.detail)) {
  return payload.detail
    .map((issue) => `${issue.loc?.at(-1)}: ${issue.msg}`)
    .join(". ");
}
```

Use try, catch, and finally

```
try {
  const response = await apiFetch(...);
  // handle success
} catch (error) {
  showToast(error.message, "error");
} finally {
  setButtonBusy(submitButton, false);
}
```

- try contains code that may fail.
- catch displays a helpful error.
- finally restores the button after success or failure.

14. Debug requests in Chrome

Follow these steps:

- Open the frontend in Chrome.
- Press F12 or right-click and choose Inspect.
- Select the Network tab.
- Submit the login form.
- Click the login request.
- Inspect Headers, Payload, Response, and Status.

For successful login, students should find:

```
Request URL:     http://127.0.0.1:8000/auth/login
Request method:  POST
Content-Type:    application/json
Status:          200 OK
Payload:         email and password JSON
Response:        access_token and token_type
```

15. Common problems and solutions

Problem: Could not reach the API

- Confirm Uvicorn is still running.
- Open http://127.0.0.1:8000/docs.
- Confirm API_BASE_URL uses port 8000.

Problem: CORS error

- Serve the frontend on port 5500 or 8080.
- Make sure the exact frontend origin is in allow_origins.
- Restart Uvicorn after changing CORS settings.

Problem: 401 Unauthorized

- Log in again and inspect localStorage.
- Check for Authorization: Bearer ... in Network -> Headers.
- Confirm the backend secret key did not change after issuing the token.

Problem: 422 Unprocessable Entity

- Compare JSON keys with the Pydantic schema.
- Check the title and description minimum lengths.
- Use only low, medium, or high for priority.

Problem: Registration returns 409

The email already exists. Use a different email or sign in to the existing account.

16. Full connection checklist

- FastAPI starts on port 8000.
- The frontend starts on port 5500.
- CORS includes the frontend origin.
- API_BASE_URL points to port 8000.
- HTML input names match Pydantic field names.
- POST requests use JSON.stringify().
- JSON requests include Content-Type: application/json.
- Login stores the returned access token.
- Protected requests include the bearer token.
- Responses are checked with response.ok.
- Errors are caught and displayed to the user.
- Ticket data is rendered safely into the page.
- Logout removes the stored token.

17. Beginner vocabulary

API

A defined way for programs to communicate.

Endpoint

An HTTP method and URL that perform one API operation.

Request

Information sent from the frontend to the backend.

Response

The backend's answer, including data and a status code.

JSON

A text format used to exchange structured data.

HTTP method

The operation type, such as GET, POST, PATCH, or DELETE.

Header

Extra request information, such as content type or authentication.

JWT

A signed token used to identify an authenticated user.

Bearer token

A token sent in the Authorization header.

CORS

Browser rules controlling communication between different origins.

async and await

JavaScript syntax for waiting for asynchronous work such as network requests.

DOM

The browser's JavaScript representation of the HTML page.

18. Student exercises

- Use Network tools to find the login method, URL, request JSON, status, and response JSON.
- Remove the token in Application -> Local Storage, refresh, and explain what happens.
- Change pageSize from 8 to 3 and observe pagination.
- Submit a ticket title shorter than five characters and study validation.
- Temporarily change the API port to 9999, explain the error, and restore port 8000.
- Add a high-priority count using /tickets?page=1&page_size=1&priority=high.
- When PATCH is implemented, design an edit request using apiFetch().

19. Final mental model

```
1. LISTEN
   JavaScript catches a browser event.

2. COLLECT
   Read and validate values from the HTML form.

3. REQUEST
   Send method, URL, headers, and JSON with apiFetch().

4. WAIT
   await the FastAPI response.

5. HANDLE
   Use successful JSON or display an error.

6. RENDER
   Update the DOM so the student sees the result.
```

NOTE: If students remember only one pattern, remember this: Event -> collect data -> apiFetch -> await response -> update DOM -> catch errors.

Quick classroom demonstration

A useful live demonstration is:

- Register a new account while the Network tab is open.
- Log in and point out the access_token response.
- Show the token in localStorage.
- Create a ticket and inspect the Authorization header.
- Filter the ticket list and inspect the query string.
- Stop FastAPI and demonstrate the network error.
- Restart FastAPI and show the application recovering.

End of guide

Examples match the Deskflow frontend and FastAPI routes.